

Authentication and Access Control

1) Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Step 1: Create the Login Fields

- Create two input fields: Username and Password.
- Use hardcoded credentials for this classroom/mock exercise.

Step 2: Check the Credentials

- Compare the entered username with the stored admin username.
- Compare the entered password with the stored admin password.
- Display "Access Granted" only when both values match.
- Otherwise, display "Access Denied".

Example Python Code

```
admin_username = "admin"
admin_password = "Admin@123"

•

• username = input("Username: ")
  password = input("Password: ")
•

  if username == admin_username and password == admin_password:
      print("Access Granted")

• else:
      print("Access Denied")
```

Expected Result

- Correct username + correct password → Access Granted.
- Incorrect username or password → Access Denied.

Important Note

- Hardcoded passwords are suitable only for a classroom/mock exercise. Real applications should store passwords securely and should not keep them directly in source code.

2) Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the resource, otherwise `False`. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

Step 1: Create the Permission Dictionary

```
permissions = {  
    "customer": ["order_history"],  
    "restaurant_owner": ["menu_edit", "order_history"],  
    "admin": ["menu_edit", "order_history", "admin_dashboard"]  
}
```

Step 2: Create the Function

```
def check_access(user_role, resource):  
    return resource in permissions.get(user_role, [])
```

Step 3: Test the Function

```
print(check_access("customer", "order_history"))  
print(check_access("customer", "menu_edit"))  
print(check_access("restaurant_owner", "menu_edit"))  
print(check_access("restaurant_owner", "admin_dashboard"))  
print(check_access("admin", "admin_dashboard"))
```

Expected Output

- customer → order_history: True.
- customer → menu_edit: False.
- restaurant_owner → menu_edit: True.
- restaurant_owner → admin_dashboard: False.
- admin → admin_dashboard: True.

Explanation

- The dictionary maps each role to its allowed resources.
- The function checks whether the requested resource exists in the role's permission list.
- If the role is unknown, access is denied by using an empty default list.

3) Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Step 1: Verify the Password

- Ask the user to enter the password.
- Continue to the OTP stage only when the password is correct.

Step 2: Generate a 6-Digit OTP

- Use Python's random module to generate a number between 100000 and 999999.
- Display the generated OTP in the console for this classroom simulation.

Example Python Code

```
import random

•

correct_password = "Paytm@123"
password = input("Enter password: ")

•

if password == correct_password:
    otp = random.randint(100000, 999999)
    print("OTP:", otp)

•

    entered_otp = int(input("Enter OTP: "))

•

    if entered_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP")

• else:
    print("Invalid Password")
```

Expected Result

- Correct password + correct OTP → Login Successful.
- Correct password + incorrect OTP → Invalid OTP.
- Incorrect password → Invalid Password.

Important Note

- This is a classroom simulation. A real OTP should be generated and delivered through a secure authentication service rather than displayed to the user in the same console.

4) Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

Step 1: Understand the Condition

- A user must satisfy both conditions: isPremium must be True and lastLogin must be within the last 7 days.
- Using 2024-06-10 as today, the seven-day window is 2024-06-04 through 2024-06-10.

Step 2: Example User Data

```
users = [  
    {"username": "alice", "isPremium": True, "lastLogin": "2024-06-10"},  
    {"username": "bob", "isPremium": False, "lastLogin": "2024-06-09"},  
    {"username": "charlie", "isPremium": True, "lastLogin": "2024-06-05"},  
    {"username": "david", "isPremium": True, "lastLogin": "2024-05-20"}  
]
```

Step 3: Create the Filtering Function

```
from datetime import datetime, timedelta  
  
def active_premium_users(users):  
    today = datetime.strptime("2024-06-10", "%Y-%m-%d").date()  
    cutoff = today - timedelta(days=6)  
  
    return [  
        user for user in users  
        if user["isPremium"]  
        and cutoff <= datetime.strptime(  
            user["lastLogin"], "%Y-%m-%d"  
        ).date() <= today  
    ]
```

Expected Result

- alice is included because she is premium and logged in on 2024-06-10.
- charlie is included because she is premium and logged in on 2024-06-05.
- bob is excluded because the account is not premium.
- david is excluded because the last login is outside the seven-day window.

5) Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Common Access-Control Mistakes

- 1. Broken authorization / IDOR — Fix: Always verify on the server that the authenticated user is authorized to access the requested object.
- 2. Privilege escalation — Fix: Enforce role and permission checks on every protected server-side action instead of trusting the client.
- 3. Relying on hidden UI elements for security — Fix: Hiding an admin button is not authorization; enforce the permission check on the backend.
- 4. Missing access checks on APIs — Fix: Apply authentication and authorization checks consistently to every sensitive API endpoint.
- 5. Overly broad permissions — Fix: Follow the principle of least privilege and give each role only the minimum access it requires.

Conclusion

- Strong access control requires authentication plus server-side authorization, least privilege, and consistent permission checks.
- Access decisions should never depend only on values supplied or controlled by the client.